Defiende el servidor

MLOps 2026-II · Universidad Externado de Colombia · Taller evaluable de versionado · Daniela González Ortiz (individual) · Grupo A · Repositorio: github.com/danielaestudio02-del/Taller_1

Cadena de trazabilidad. Código (commit Git) → Datos (`data/*.dvc`, md5) → Experimento (run ID de MLflow) → Modelo (revisión SHA en Hugging Face). Cada paquete de modelo lleva en `config.json` y en su model card el commit, las referencias DVC, el run ID y el SHA-256 del artefacto. `predict.py` rechaza cargar un artefacto cuyo hash no coincida.

1. Datos y limpieza

Se usan las particiones publicadas de UNSW-NB15: `UNSW_NB15_training-set.csv` (175.341 filas, 45 columnas) y `UNSW_NB15_testing-set.csv` (82.332 filas). Se descargaron el 30-10-2026 del mirror fijado de la guía, y sus SHA-256 coinciden con el anexo (`bec7dd5e…` y `734fe664…`). Los archivos traen una BOM UTF-8 en la cabecera, que se maneja con `utf-8-sig`. No hay valores faltantes. El objetivo es `label`, con ataque = 1 como clase positiva. Entrenamiento tiene 119.341 ataques y 56.000 normales (68 % de ataques); test tiene 45.332 ataques y 37.000 normales (55 %). Se excluyen `id` (identificador), `attack_cat` (revela la respuesta) y `label`. Quedan 42 predictoras: 39 numéricas y 3 categóricas (`proto`, `service`, `state`).

Duplicados y contradicciones. En el archivo de entrenamiento solo hay 101.040 entradas distintas, así que el 42,4 % de las filas (74.301) repite unas variables predictoras ya vistas. Además, 229 grupos de entradas idénticas (940 filas) tienen etiquetas contradictorias; en test son 6 grupos (14 filas). **Política:** no se eliminan filas, porque los duplicados reflejan la frecuencia real del tráfico y las contradicciones son ruido irreducible. Lo que sí se impone es que todas las filas con entradas idénticas queden en el mismo split.

2. Particiones y prevención de fuga

Entrenamiento y validación salen solo del archivo de entrenamiento, con `StratifiedGroupKFold(5, shuffle, semilla 42)`: el pliegue 0 es validación. El grupo es el hash de las 42 predictoras y la estratificación es por `label`. Resultado: 140.274 filas de train y 35.067 de validación (23.867 ataques), con 0 grupos compartidos. Las huellas de contenido y de `row_id` de cada split están en `auditoria.json`. Las particiones son reproducibles: al repetir `prepare.py`, `dvc status` no muestra cambios. Imputación, codificación y escalado viven dentro de un `Pipeline` de scikit-learn ajustado solo con train y se guardan junto al modelo. **Cruces con test:** 8.541 filas de test (10,4 %) tienen entradas idénticas a alguna fila de train o validación, y los niveles `ACC` y `CLO` de `state` no aparecen en train. Por eso el test no es totalmente independiente y conviene leer sus métricas con cautela.

Versiones de datos en DVC. v1 son los CSV recibidos (`data/raw.dvc`, md5 `fe93051a…`, commit `1b826c1f`). v2 son los datos preparados con `row_id`, `group_id` y `split`, más un manifiesto (`data/prepared.dvc`, md5 `53545c81…`, commit `9a17cd22`). El almacenamiento remoto es el repositorio *dataset* público `Daniela201000/defiende-el-servidor-dvc` de Hugging Face (remote `hfstore`).

3. Cuatro experimentos (validación, umbral 0,5)

Todos usan las mismas particiones y la semilla 42, y se entrenaron en el commit `9a17cd22` con el árbol de trabajo limpio; `train.py` se niega a entrenar si hay cambios sin commit. Cada run registra en MLflow la hipótesis, el cambio, los parámetros, el commit, el md5 DVC, las métricas, la matriz de confusión, el modelo completo con su SHA-256 y las puntuaciones de validación. Cuando un denominador es 0, la métrica se reporta como 0,0 (no ocurrió en ningún caso).

ID · run	Familia · cambio · hipótesis	Recall	Precision	FPR	Macro-F1
exp1 · `d918b875`	Lineal. Regresión logística (C=1) con one-hot y escalado. Referencia sencilla.	0,9886	0,9223	0,1776	0,9225

exp2 · `420be7f7`	Lineal. log1p en las numéricas + C=0,1: las colas pesadas perjudican al modelo lineal.	0,9771	0,9354	0,1437	0,9274
exp3 · `831fc6f5`	Árboles. Random Forest (150 árboles, profundidad 20): capta interacciones no lineales.	0,9808	0,9546	0,0994	0,9476
exp4 · `a5ca5470`	Árboles. HistGradientBoosting (300 iteraciones, lr 0,1): el boosting corrige residuos.	0,9764	0,9611	0,0842	0,9502

En las dos familias, los cambios confirman sus hipótesis. log1p reduce la FPR del modelo lineal (de 0,178 a 0,144), pero el límite lineal sigue generando muchas falsas alarmas. Los árboles bajan la FPR a menos de la mitad sin perder mucho recall. exp4 obtiene el mejor macro-F1 con un modelo de 1,1 MB, mientras que el bosque ocupa 44 MB.

4. v1 y v2: qué cambia y por qué

	v1	**v2**
Modelo · umbral	exp1, regresión logística · 0,50	exp4, HistGradientBoosting · 0,65
Commit de la solución	`330015e9da6c114e870f444c720d585952fbfaac`	`2ae6479c9528c7dcc2342908618d4b432c8131fb`
SHA-256 de model.joblib	`2b350cdc…a8ab09c6c`	`ac60460c…69d5f2b62`
Revisión HF (Daniela201000/defiende-el-servidor)	`691479df2d72c01e99bd1b32c9d5b82368ff8b70`	`e2f7bfdd932f383d7cf40c5413d5d407408889ba`
Validación: recall / precision / FPR / macro-F1	0,9886 / 0,9223 / 0,1776 / 0,9225	0,9540 / 0,9760 / 0,0499 / 0,9463
Matriz de validación (TN FP / FN TP)	9.211 1.989 / 272 23.595	10.641 559 / 1.098 22.769

v1 es la primera solución funcional. v2 cambia la **familia de modelo y el umbral**. **Mejora:** las falsas alarmas sobre tráfico normal bajan de 1.989 a 559 (FPR de 17,8 % a 5,0 %) y suben la precisión y el macro-F1. **Empeora:** el recall de ataque baja de 0,989 a 0,954 (826 ataques más sin detectar). Por categoría (`errores_validacion_por_categoria.json`, que solo sirve de interpretación), v2 detecta el 73 % de *Fuzzers* (v1 detecta el 95 %) y el 87 % de *Analysis* (v1, el 99 %). En cambio, mantiene o mejora DoS, Exploits, Generic, Reconnaissance, Backdoor y Worms. Se aceptan esos errores porque una FPR del 18 % genera una alarma por cada cinco conexiones normales, y eso agota la capacidad de respuesta del equipo de seguridad. Los Fuzzers se parecen mucho al tráfico normal y son el principal costo de esta decisión.

5. Elección del umbral (solo validación)

La regla se fijó en `configs/selection.yaml` antes de evaluar test. El modelo es el de mayor macro-F1 de validación con umbral 0,5 (exp4). El umbral, buscado en una rejilla de 0,05 a 0,95 con paso 0,01, es el que maximiza el recall de ataque sujeto a FPR ≤ 0,05, el límite operativo de falsas alarmas; da 0,65. La curva está en `seleccion_umbral.csv`. Regla de decisión: ataque si `score_attack ≥ 0,65`, donde score es `predict_proba[:,1]`. La selección se congeló en el commit `2ae6479c`, anterior al commit de evaluación `701a209c`, y `evaluate_test.py` se niega a correr si `seleccion.json` no está versionado.

6. Test final (evaluación única de v2 congelada)

Filas	**Recall**	**Precision**	**FPR**	**Macro-F1**	**Accuracy**	**TN · FP · FN · TP**
82.332	0,9701	0,8681	0,1806	0,8996	0,9024	30.319 · 6.681 · 1.357 · 43.975

Se evaluó todo el archivo publicado, sin quitar filas, con el artefacto `ac60460c…`. Las predicciones por fila (`row_id = test:<id>`, `y_true`, `score_attack`, `y_pred`) están en `predicciones-test.csv`, y `recompute_metrics.py` reproduce las métricas exactamente. **El resultado empeora frente a validación y se conserva tal cual:** el recall se mantiene (0,970), pero la FPR sube de 0,050 a 0,181. El tráfico normal del archivo de

test es distinto del de entrenamiento, que es un cambio de distribución conocido entre las particiones de UNSW-NB15. Además, test tiene menos ataques (55 % frente a 68 %), lo que reduce la precisión. Con el umbral fijado en validación, el límite de FPR del 5 % no se cumple en test. No se reajustó nada: hacerlo con test invalidaría su carácter de evaluación final. Una v3 debería recalibrar el umbral con datos normales más recientes o más representativos.

7. Recuperación

`scripts/recuperacion.sh` clona el commit elegido en una carpeta nueva, crea un venv nuevo con las dependencias fijadas y usa una caché DVC y un `HF_HOME` vacíos. Después ejecuta `dvc pull` desde el remote entregado y verifica los SHA-256 de los crudos y las huellas de los splits. Descarga v1 y v2 de Hugging Face por SHA completo, predice sobre 25 filas fijas de validación (`muestra_validacion.csv`) y compara con las predicciones guardadas antes de publicar, exigiendo etiquetas iguales y una diferencia de score ≤ 1e-9. **Resultado (30-10-2026, commit `3da339d1`): OK.** `dvc pull` trajo 8 archivos desde Hugging Face con huellas coincidentes. Las 25 etiquetas de v1 y de v2 coinciden, con una diferencia máxima de score de 3,7e-11 (v1) y 4,9e-11 (v2), y el SHA-256 de cada artefacto descargado coincide con el registrado. El log está en `evidencias/recuperacion/`.

8. Limitaciones

- El dataset es histórico (2015) y de laboratorio: no demuestra eficacia sobre una red actual. La FPR de test indica que el umbral no se transfiere bien a otro tráfico normal.
- El 10,4 % de las entradas de test ya se vio en train/val, así que el test es parcialmente optimista. Las contradicciones fijan un error mínimo.
- v2 detecta peor los *Fuzzers*. El umbral optimiza recall con una restricción de FPR, no un costo explícito por tipo de error.
- MLflow es local (sqlite) y se entrega como exportación. `model.joblib` usa pickle, así que solo deben cargarse revisiones verificadas por hash.

Referencias. Moustafa y Slay (2015), UNSW-NB15, MilCIS · UNSW, research.unsw.edu.au/projects/unsw-nb15-dataset · Documentación de scikit-learn (common pitfalls), DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión, model cards). Uso de IA: Claude Code (Anthropic) para código, ejecución y redacción, verificado según `entrega.md`.